

Practice Guide 2: Linear Models, fastText and Small Language Models

Aprendizagem Aplicada à Segurança

Mário Antunes

October 2, 2026

Objective

Climb the second half of the spam ladder. You will train **logistic regression** by gradient descent with JAX, a **fastText** classifier with character n-grams, and a **linear probe on frozen embeddings** of a small language model, and you will find out **when** each one is the right choice.

- **Notebook to complete:** `guide_02.ipynb`. Cells marked TODO are yours; check cells validate your work.
- **Requirements:** the base environment plus the `slm` group: `pip install --group slm --extra-index-url https://download.pytorch.org/whl/cpu` (the first run downloads a 90 MB model).
- **Time:** about 2 hours. **Deliverable:** the completed notebook and the answers below.
- **Prerequisite:** Guide 1 (tokenizer, Naive Bayes, TF-IDF).

Background

Logistic regression. $z = \mathbf{w} \cdot \mathbf{x} + b$, $P(\text{spam}) = \sigma(z)$. Cross-entropy with L2: $J = -\frac{1}{n} \sum_i [y_i \log \sigma(z_i) + (1 - y_i) \log \sigma(-z_i)] + \frac{\lambda}{2} \|\mathbf{w}\|^2$, gradient $\frac{1}{n} X^\top (\sigma(X\mathbf{w} + b) - \mathbf{y}) + \lambda \mathbf{w}$. Never compute $\log(\text{sigmoid}(z))$ directly: for $z = -800$ it is $-\infty$; use `jax.nn.log_sigmoid`. Gradient descent: $\theta \leftarrow \theta - \eta \nabla J$.

fastText. A word vector is the sum of the vectors of its character n-grams (<fr, fre, ree, ee>); the message vector is the average over words and word bigrams; a linear layer with softmax classifies. Misspelt words share most n-grams with the correct word.

SLM embeddings. A pre-trained Transformer maps each message to a dense vector. The encoder stays **frozen**; only a small classifier is trained. Embeddings are not centred,

so standardise each dimension (with statistics from the training set) before the linear probe.

Tasks

Part	What you do	Check
A	Data and TF-IDF features (given)	
B	Implement the stable loss ; one gradient-descent update; read the weights	loss and gradient OK
C	Train fastText ; nearest neighbours of a misspelt word; leet-speak experiment	Questions C1, C2
D	Encode with MiniLM ; standardise; linear probe	F1 close to 0.95
E	Learning curve : TF-IDF versus embeddings with 20, 50, 100, 500 labels	table
F	Summary table and recommendation	Question F1

Questions for the report

1. **(B1)** Plot the loss history for $\eta = 1, 50$ and 500 . Describe and explain each behaviour.
2. **(B2)** The weight vector is the gradient of the score with respect to the input. Which 10 words push a message towards *ham* the most? Describe how an attacker could exploit them, and what they would need to know.
3. **(C1)** *freee* never appears in the training set. Why does fastText still give it a meaningful vector? What would a plain bag-of-words model do?
4. **(C2)** Report the recall of fastText and of the TF-IDF logistic regression on the *leet-speak* spam ($a \rightarrow 4, e \rightarrow 3, o \rightarrow 0$). Explain the difference.
5. **(D)** Why do we standardise the embeddings, and why fit the scaler on the training set only?
6. **(E)** In which range of labelled examples do the embeddings win, and by how much? Why do they need fewer labels?
7. **(F1)** Fill in the table and choose a model for a mail gateway that processes 10,000 messages per second and receives few new labels. Justify with cost per message, accuracy and robustness.

Model	Precision	Recall	F1	cost per message
TF-IDF + logistic regression (GD)				
fastText				

Model	Precision	Recall	F1	cost per message
MiniLM embeddings + LR				

Optional challenges

- Replace MiniLM by Qwen/Qwen3-Embedding-0.6B (600 M parameters, about 5 minutes on a CPU). Is it worth the cost?
- Add a **Keras** head (one hidden layer of 64 units) on the embeddings (KERAS_BACKEND=jax) and compare with the linear probe.
- Repeat part C with random character noise (deleting or doubling letters) instead of leet-speak.

Grading

Working implementations that pass the checks (60%), the completed table and learning curve (20%), the answers to the questions (20%). Reference solutions: [solution/](#).